

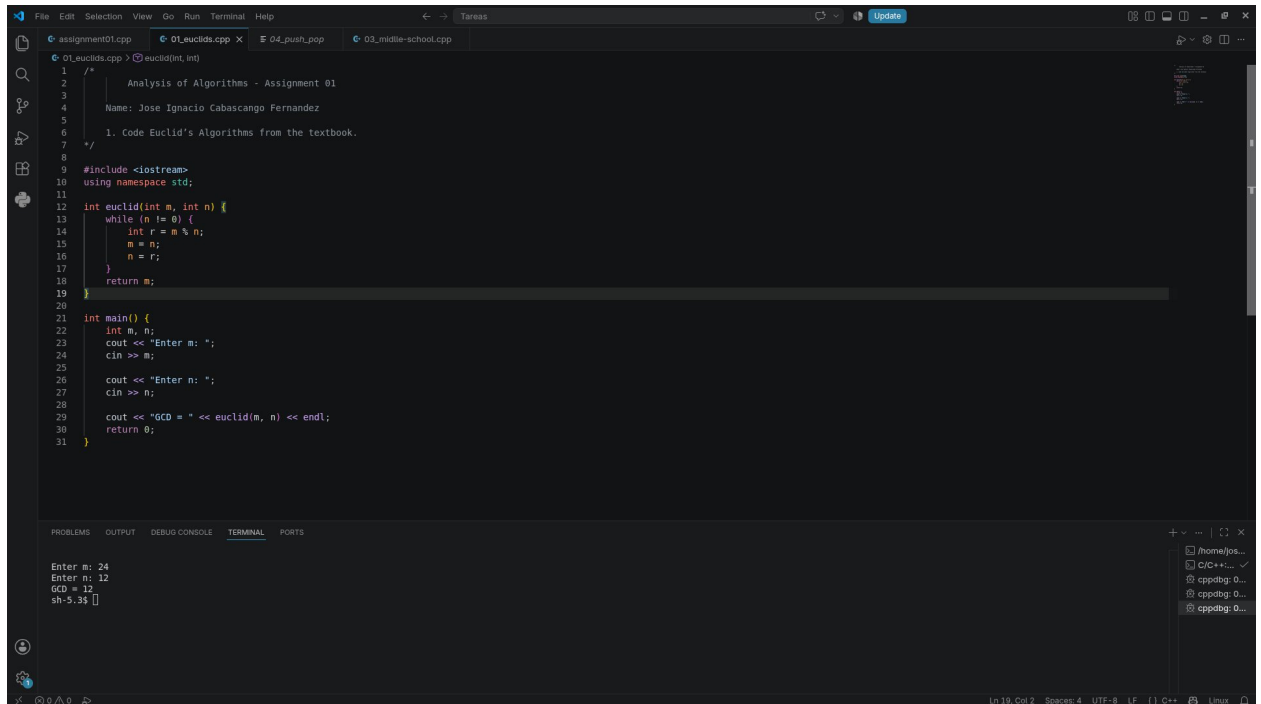
Analysis of Algorithms

Assignment 01

Name: Jose Ignacio Cabascango Fernandez

Link GitHub: <https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignment-01/src>

1. Code Euclid's Algorithms from the textbook.



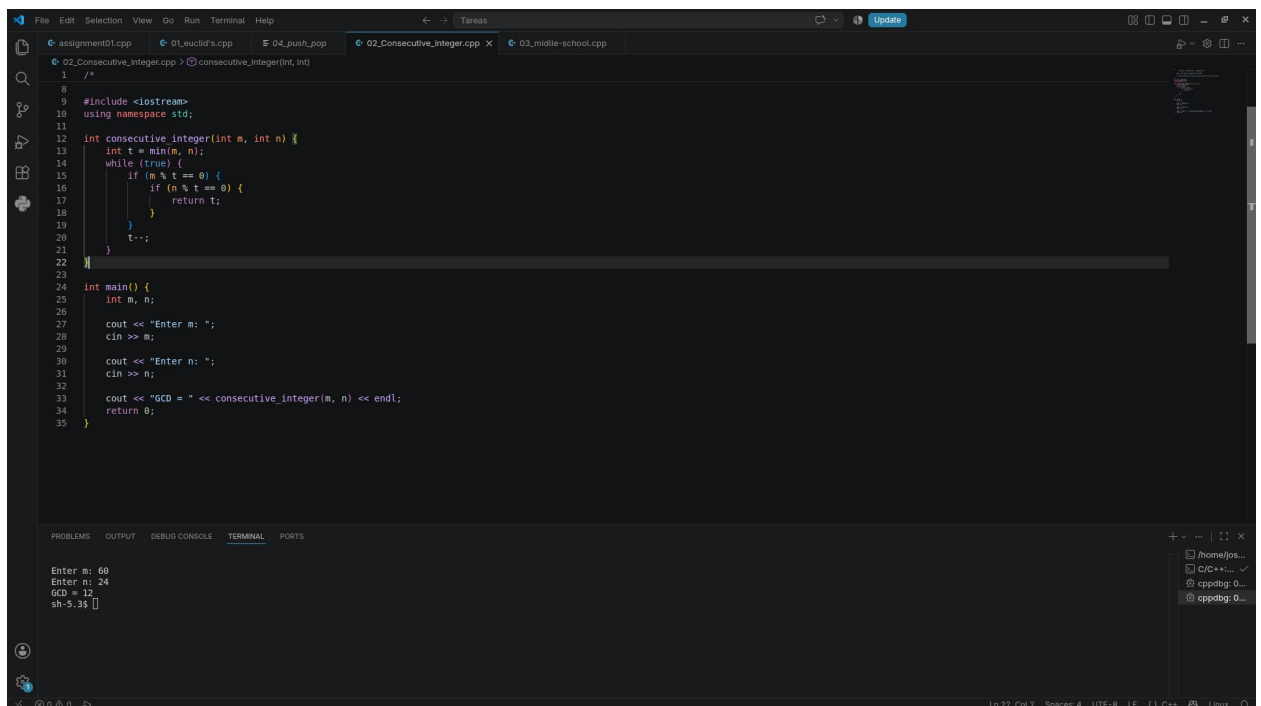
The screenshot shows a C++ IDE with a file named `01_euclids.cpp`. The code implements Euclid's algorithm to find the Greatest Common Divisor (GCD) of two numbers. The `main` function prompts the user to enter two numbers, `m` and `n`, and then prints the result of `euclid(m, n)`. The `euclid` function uses a `while` loop to repeatedly calculate the remainder of `m` divided by `n` until it reaches zero, at which point `n` is the GCD.

```
1  /*
2     Analysis of Algorithms - Assignment 01
3
4     Name: Jose Ignacio Cabascango Fernandez
5
6     1. Code Euclid's Algorithms from the textbook.
7  */
8
9  #include <iostream>
10 using namespace std;
11
12 int euclid(int m, int n) {
13     while (n != 0) {
14         int r = m % n;
15         m = n;
16         n = r;
17     }
18     return m;
19 }
20
21 int main() {
22     int m, n;
23     cout << "Enter m: ";
24     cin >> m;
25
26     cout << "Enter n: ";
27     cin >> n;
28
29     cout << "GCD = " << euclid(m, n) << endl;
30     return 0;
31 }
```

The terminal output shows the execution of the program:

```
Enter m: 24
Enter n: 12
GCD = 12
sh-5.3$
```

2. Code Consecutive integer checking algorithm from the textbook.



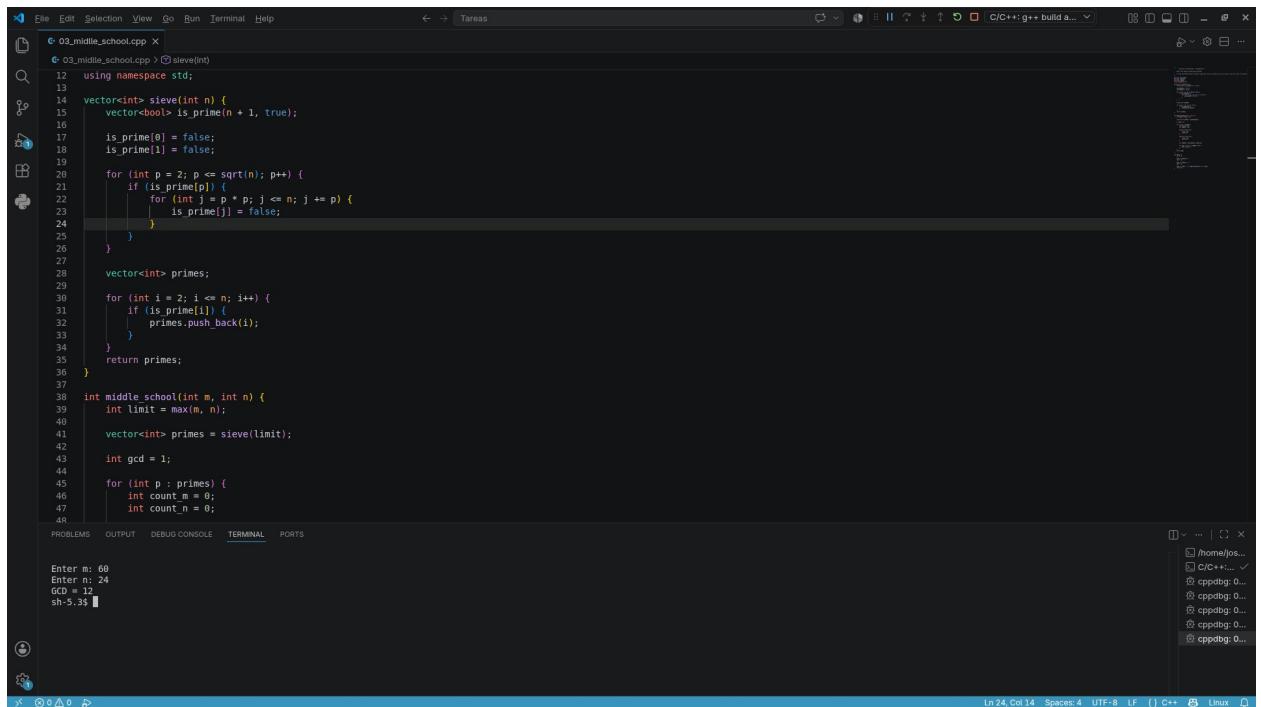
The screenshot shows a C++ IDE with a file named `02_Consecutive_Integer.cpp`. The code implements an algorithm to find the Greatest Common Divisor (GCD) of two numbers by checking consecutive integers. The `main` function prompts the user to enter two numbers, `m` and `n`, and then prints the result of `consecutive_integer(m, n)`. The `consecutive_integer` function starts with `t = min(m, n)` and uses a `while` loop to check if `t` divides both `m` and `n`. If it does, it returns `t`; otherwise, it decrements `t` and continues the loop.

```
1  /*
2
3
4
5
6
7
8
9  #include <iostream>
10 using namespace std;
11
12 int consecutive_integer(int m, int n) {
13     int t = min(m, n);
14     while (true) {
15         if (m % t == 0) {
16             if (n % t == 0) {
17                 return t;
18             }
19         }
20         t--;
21     }
22 }
23
24 int main() {
25     int m, n;
26
27     cout << "Enter m: ";
28     cin >> m;
29
30     cout << "Enter n: ";
31     cin >> n;
32
33     cout << "GCD = " << consecutive_integer(m, n) << endl;
34     return 0;
35 }
```

The terminal output shows the execution of the program:

```
Enter m: 60
Enter n: 24
GCD = 12
sh-5.3$
```

3. Code the Middle-school procedure algorithm from the textbook (You will need to code the Sieve of Eratosthenes algorithm too)



```
12 using namespace std;
13
14 vector<int> sieve(int n) {
15     vector<bool> is_prime(n + 1, true);
16
17     is_prime[0] = false;
18     is_prime[1] = false;
19
20     for (int p = 2; p <= sqrt(n); p++) {
21         if (is_prime[p]) {
22             for (int j = p * p; j <= n; j += p) {
23                 is_prime[j] = false;
24             }
25         }
26     }
27
28     vector<int> primes;
29     for (int i = 2; i <= n; i++) {
30         if (is_prime[i]) {
31             primes.push_back(i);
32         }
33     }
34     return primes;
35 }
36
37 int middle_school(int m, int n) {
38     int limit = max(m, n);
39
40     vector<int> primes = sieve(limit);
41
42     int gcd = 1;
43
44     for (int p : primes) {
45         int count_m = 0;
46         int count_n = 0;
47     }
48 }
```

Enter m: 60
Enter n: 24
GCD = 12
sh-5.33

4. Find the way C++ uses to push and pop elements to and from a list to behave like a stack. Create an example that showcases the idea.

For the creation of a stack with the help of a list, we use `std::list`, which can be used to represent a stack. We must use the LIFO (Last in, First out) principle. To achieve this, we use the operations `push_front()` and `pop_front()`.

- `push_front()` adds an element to the beginning of the list.
- `pop_front()` removes the first element from the list.
- `front()` allows you to check the first element without removing it.
- `empty()` allows checking if the list is empty.

For example, if the elements 5, 6, 7, and 8 are added using `push_front()`, the last element added, 8, remains at the top of the stack. When using `pop_front()`, the element 8 will be the first to exit.

Both `push_front()` and `pop_front()` have a time complexity of $O(1)$ in `std::list`.

Example:

The screenshot shows a C++ IDE with a file explorer on the left and a terminal at the bottom. The main editor displays the code for `04_push_pop.cpp`. The code implements a stack using `std::list`. It pushes elements 5, 6, 7, and 8 onto the stack. Then, it prints the stack contents (8 7 6 5), pops the top element (8), and prints the stack again (7 6 5). The terminal output matches the code's execution.

```
1  /*  
2  */  
3  
4  #include <iostream>  
5  #include <list>  
6  using namespace std;  
7  
8  
9  int main() {  
10     list<int> stack;  
11  
12     // push  
13     stack.push_front(5);  
14     stack.push_front(6);  
15     stack.push_front(7);  
16     stack.push_front(8);  
17  
18     cout << "Insert element: " << stack.front() << endl;  
19  
20     // print the stack before pop  
21     cout << "Stack: ";  
22     for (int i : stack) {  
23         cout << i << " ";  
24     }  
25     cout << endl;  
26  
27     // pop  
28     stack.pop_front();  
29     cout << "After pop: " << stack.front() << endl;  
30  
31     // print the stack after pop  
32     cout << "Stack: ";  
33     for (int i : stack) {  
34         cout << i << " ";  
35     }  
36  
37     cout << endl;  
38     return 0;  
39 }
```

Terminal Output:

```
Insert element: 8  
Stack: 8 7 6 5  
After pop: 7  
Stack: 7 6 5  
sh-5.3$
```

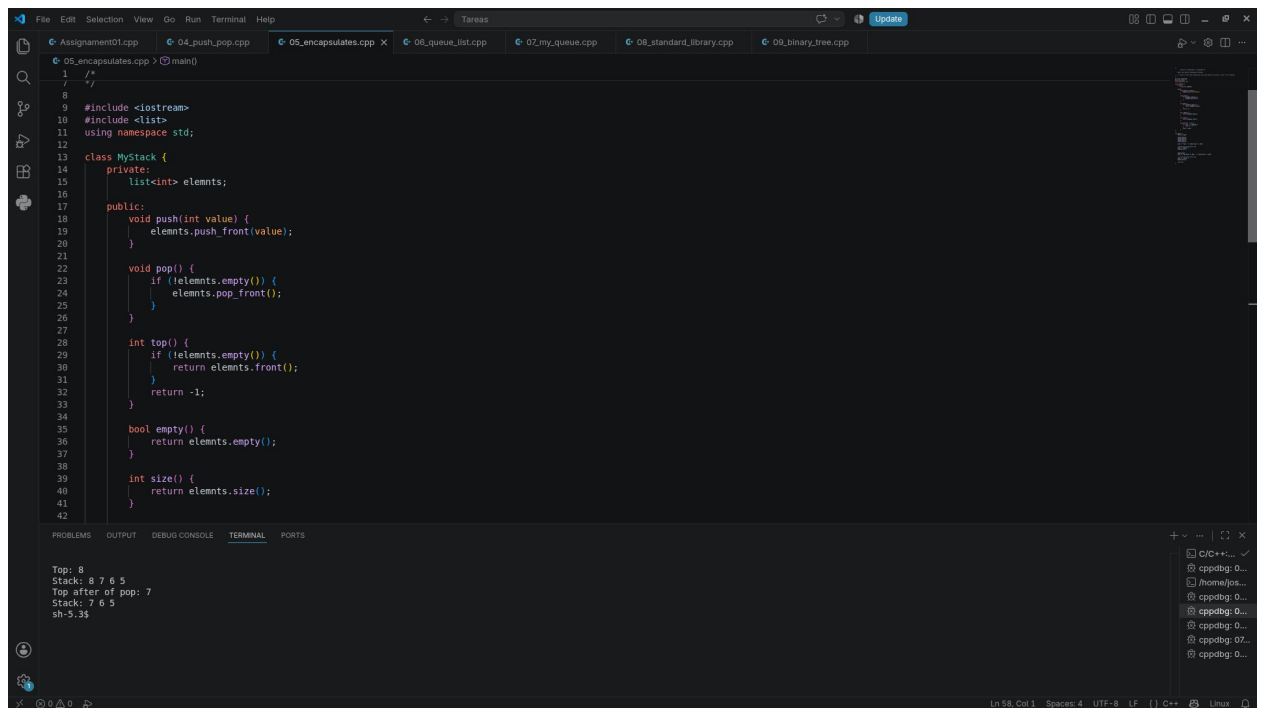
5. Create a class that encapsulates the stack behavior describe in point 4 (Ex: MyStack)

To encapsulate the behavior of a stack, a class called `MyStack` can be created. The class internally contains a `std::list`, while its operations are exposed through methods such as:

- `push()` to add elements.
- `pop()` to remove the top element.
- `top()` to query the top element.
- `empty()` to check if the stack is empty.
- `size()` to know the number of elements.

The objective of encapsulating the structure is to prevent external code from directly manipulating the list. Thus, the internal implementation remains protected and the user only needs to know the stack operations.

Example:



```
1  /*
2  */
3
4  #include <iostream>
5  #include <list>
6  using namespace std;
7
8  class MyStack {
9  private:
10     list<int> elemnts;
11
12 public:
13     void push(int value) {
14         elemnts.push_front(value);
15     }
16
17     void pop() {
18         if (!elemnts.empty()) {
19             elemnts.pop_front();
20         }
21     }
22
23     int top() {
24         if (!elemnts.empty()) {
25             return elemnts.front();
26         }
27         return -1;
28     }
29
30     bool empty() {
31         return elemnts.empty();
32     }
33
34     int size() {
35         return elemnts.size();
36     }
37
38
39
40
41
42 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Top: 8
Stack: 8 7 6 5
Top after of pop: 7
Stack: 7 6 5
sh-5.3\$

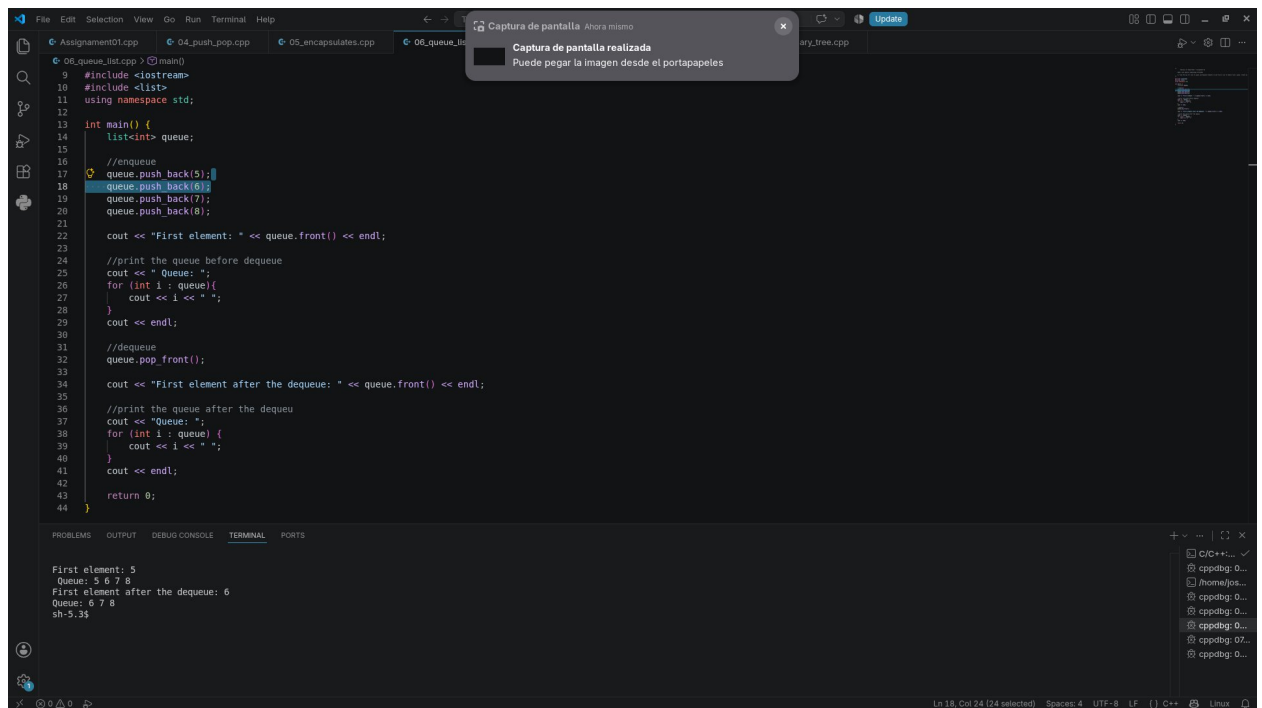
6. Find the way C++ uses to queue and dequeue elements to and from a list to behave like a queue. Create an example that showcases the idea.

For the creation of a queue with the help of a list, we use `std::list`, which can be used for the creation of a queue. We must use the FIFO principle (First In, First Out). To achieve this, we use the `push_back()` and `pop_front()` operations. The main operations are:

- `push_back()` adds an element to the end.
- `pop_front()` removes the first element.
- `front()` allows querying the first element
- `back()` allows you to query the last element.
- `empty()` checks if the queue is empty.

For example, if the elements 5, 6, 7, and 8 are added using `push_back()`, the last element added, 8, remains at the end of the queue. When using `pop_front()`, the element 5 will be the first to come out.

Example:



```
08_queue_test.cpp > main()
9  #include <iostream>
10 #include <list>
11 using namespace std;
12
13 int main() {
14     list<int> queue;
15
16     //enqueue
17     queue.push_back(5);
18     queue.push_back(6);
19     queue.push_back(7);
20     queue.push_back(8);
21
22     cout << "First element: " << queue.front() << endl;
23
24     //print the queue before dequeue
25     cout << "Queue: ";
26     for (int i : queue){
27         cout << i << " ";
28     }
29     cout << endl;
30
31     //dequeue
32     queue.pop_front();
33
34     cout << "First element after the dequeue: " << queue.front() << endl;
35
36     //print the queue after the dequeu
37     cout << "Queue: ";
38     for (int i : queue) {
39         cout << i << " ";
40     }
41     cout << endl;
42     return 0;
43 }
```

First element: 5
Queue: 5 6 7 8
First element after the dequeue: 6
Queue: 6 7 8
sh-5.3\$

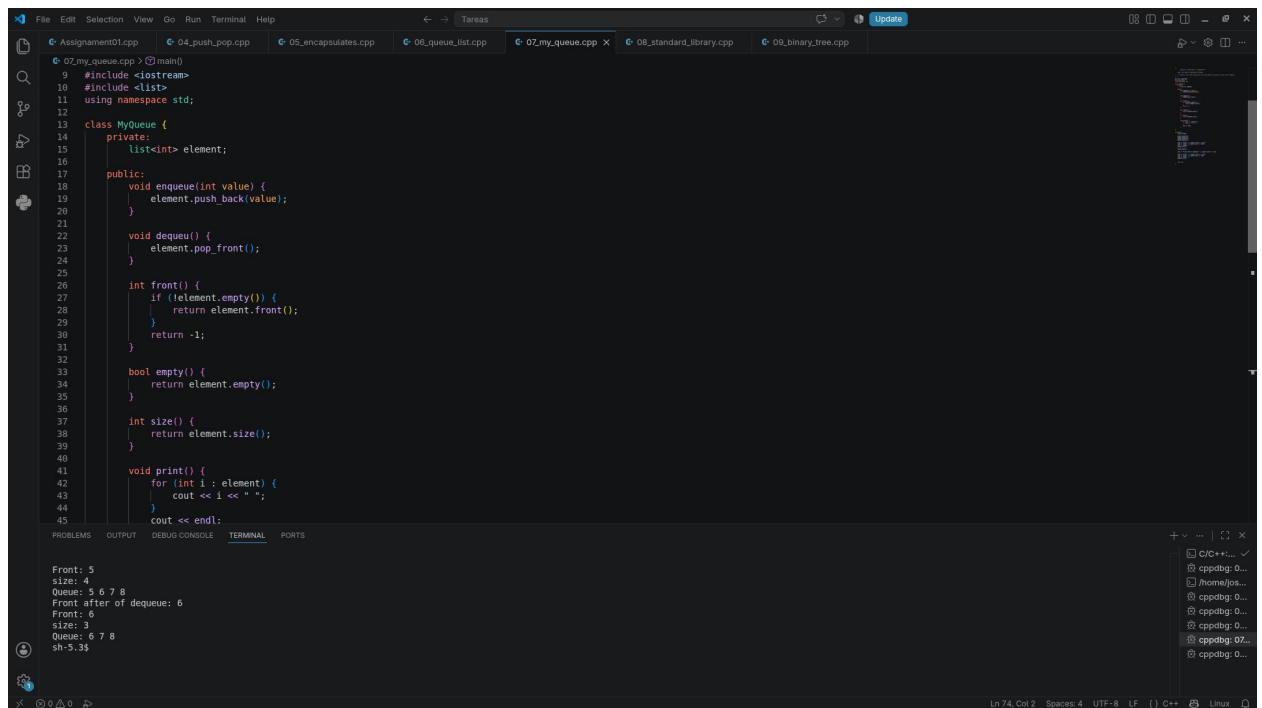
7. Create a class that encapsulates the stack behavior describe in point 6 (Ex: MyQueue)

The behavior of a queue can also be encapsulated using a custom class called MyQueue. This class internally uses a `std::list` and provides methods such as:

- `enqueue()` to add elements.
- `dequeue()` to remove the first element.
- `front()` to check the first element
- `empty()` to check if it is empty.
- `size()` to get the number of elements.

The advantage of creating MyQueue is that the internal implementation is separated from the way the structure is used. The main difference between a stack and a queue is the way elements are removed.

Example:



```
9 #include <iostream>
10 #include <list>
11 using namespace std;
12
13 class MyQueue {
14 private:
15     list<int> element;
16
17 public:
18     void enqueue(int value) {
19         element.push_back(value);
20     }
21
22     void dequeue() {
23         element.pop_front();
24     }
25
26     int front() {
27         if (!element.empty()) {
28             return element.front();
29         }
30         return -1;
31     }
32
33     bool empty() {
34         return element.empty();
35     }
36
37     int size() {
38         return element.size();
39     }
40
41     void print() {
42         for (int i : element) {
43             cout << i << " ";
44         }
45         cout << endl;
46     }
47 }
```

Front: 5
size: 4
Queue: 5 6 7 8
Front after of dequeue: 6
Front: 6
size: 3
Queue: 6 7 8
sh-5.3\$

8. Explore which alternatives does C++ Standard Library provide to deal with stacks and queues. Provide examples.

The C++ Standard Library provides specialized structures for working with stacks and queues. These structures are known as container adaptors because they provide a specialized interface over an underlying container.

std::stack

std::stack represents a stack and uses the LIFO principle. Its main operations are:

- push() to insert.
- pop() to remove the top element.
- top() to query the top element.
- empty() to check if it is empty.
- size() to get the number of elements.

By default, std::stack uses std::deque as its internal container, although it can also use other compatible containers, such as std::vector or std::list.

std::queue

std::queue represents a queue and uses the FIFO principle. Its main operations are:

- push() to add an element to the end.
- pop() to remove the element from the front.
- front() to query the first element.
- back() to check the last element.
- empty() to check if it is empty.
- size() to get the number of elements.

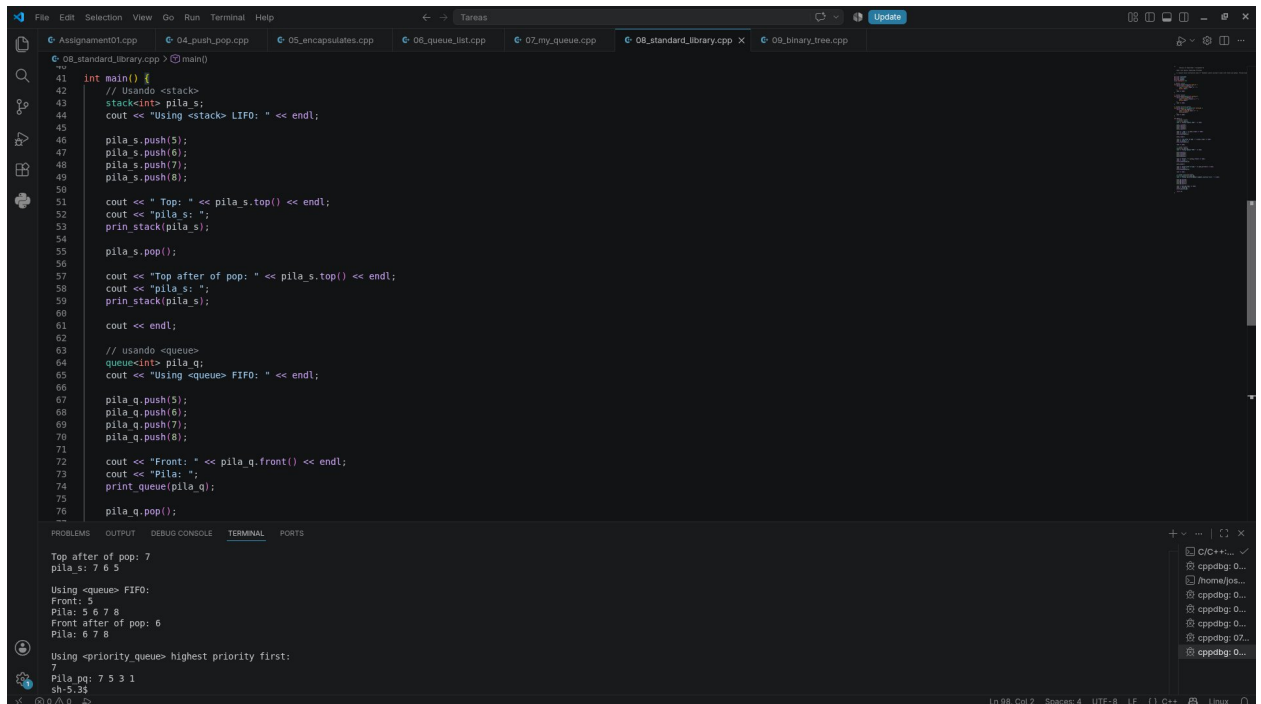
By default, std::queue uses std::deque as the underlying container, although it can also use std::list.

std::priority_queue

A priority_queue organizes its elements according to their priority. In this case, the element with the highest value has the highest priority and is available through top().

For example, if the values 10, 50, and 20 are added, the element available at the top will be 50. This allows the structure to be used in situations where the elements should not be processed simply according to the order in which they were inserted.

Example:



```
41 int main() {
42     // Using <stack>
43     stack<int> pila_s;
44     cout << "Using <stack> LIFO: " << endl;
45
46     pila_s.push(5);
47     pila_s.push(6);
48     pila_s.push(7);
49     pila_s.push(8);
50
51     cout << "Top: " << pila_s.top() << endl;
52     cout << "pila_s: ";
53     print_stack(pila_s);
54
55     pila_s.pop();
56
57     cout << "Top after of pop: " << pila_s.top() << endl;
58     cout << "pila_s: ";
59     print_stack(pila_s);
60
61     cout << endl;
62
63     // usando <queue>
64     queue<int> pila_q;
65     cout << "Using <queue> FIFO: " << endl;
66
67     pila_q.push(5);
68     pila_q.push(6);
69     pila_q.push(7);
70     pila_q.push(8);
71
72     cout << "Front: " << pila_q.front() << endl;
73     cout << "pila_q: ";
74     print_queue(pila_q);
75
76     pila_q.pop();
77 }
```

Terminal Output:

```
Top after of pop: 7
pila_s: 7 6 5

Using <queue> FIFO:
Front: 5
Pila: 5 6 7 8
Front after of pop: 6
Pila: 6 7 8

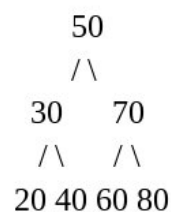
Using <priority_queue> highest priority first:
Pila pq: 7 5 3 1
sh-5.3$
```

9. Create your own Binary Tree data structure and test the insertion of elements. Insertion will need its own algorithm, research on how to implement it.

A Binary Tree is a data structure formed by nodes. Each node can have at most two children: a left child and a right child. A particular case is the Binary Search Tree (BST). In this type of tree, a rule is maintained that facilitates the search and insertion of elements. For a given node:

- Smaller values are placed in the left subtree.
- Values greater than the node are placed in the right subtree.
- Each of the subtrees must follow the same rule.

For example, if the values 50, 30, 70, 20, 40, 60, 80 are inserted, the resulting tree will be:



Example:

```
File Edit Selection View Go Run Terminal Help
09_binary_tree.cpp 04_assignment01.cpp 04_push_pop.cpp 05_encapsulates.cpp 06_queue_list.cpp 07_my_queue.cpp 08_standard_library.cpp 09_binary_tree.cpp X
class BinaryTree {
private:
    struct Node {
        int data;
        Node* left;
        Node* right;
    };
    Node(int value) {
        data = value;
        left = nullptr;
        right = nullptr;
    };
    Node* root;
    Node* insert(Node* node, int value) {
        // if we find an empty position
        if (node == nullptr) {
            return new Node(value);
        }
        // go to the left
        if (value < node->data) {
            node->left = insert(node->left, value);
        }
        // go to the right
        else {
            node->right = insert(node->right, value);
        }
        return node;
    }
    void inorder(Node* node) {
        if (node == nullptr) {
            return;
        }
    }
};

int main() {
    BinaryTree tree;
    tree.insert(tree.root, 5);
    tree.insert(tree.root, 6);
    tree.insert(tree.root, 7);
    tree.insert(tree.root, 8);
    tree.insert(tree.root, 9);
    tree.insert(tree.root, 10);
    tree.insert(tree.root, 11);
    tree.insert(tree.root, 12);
    tree.insert(tree.root, 13);
    tree.insert(tree.root, 14);
    tree.inorder(tree.root);
    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Tree in order: 5 6 7 8 9 10 11 12 13 14
sh-5.3\$

Ln 87, Col 2 Spaces: 4 UTF-8 LF C++ Linux